TECHNICAL REPORT / v1.0.0

Heart Attack Risk Predictor

A bilingual deep-learning web application that estimates heart-disease risk from 25 clinical, laboratory and lifestyle signals, with batch scoring, what-if explanations and a built-in data explorer.

Python | Streamlit | Keras / TensorFlow | scikit-learn | Plotly | Docker

9,000	25	31	89.6%
patient records	input features	model inputs	test accuracy

Mahan Liaghatmand

Author and maintainer | November 2026 | MIT License

EDUCATIONAL USE ONLY

Not a medical device. Not clinically validated. Never use it to diagnose, treat or make decisions about a real patient.

Contents

01 Executive Summary . . . 3
02 Background and Objectives . . . 4
03 Project Journey . . . 4
04 Dataset . . . 5
05 System Architecture . . . 6
06 Feature Engineering and Model . . . 7
07 Application . . . 9
08 Explainability: What-If Analysis . . . 9
09 Engineering Quality . . . 10
10 Deployment and Operations . . . 11
11 Risks and Limitations . . . 12
12 Roadmap . . . 13
13 Appendices . . . 15

How to read this report

Sections 1-3 give the context and the story of the project. Sections 4-8 document the data, architecture, model and application in technical depth. Sections 9-11 cover engineering quality, deployment and an honest account of limitations and risks. Section 12 lays out a proposed roadmap, and the appendices collect reference material.

01 Executive Summary

Heart Attack Risk Predictor is an end-to-end machine-learning project: a small Keras neural network trained on 9,000 patient records, wrapped in a polished **Streamlit** web application. A user enters a patient profile, or uploads a CSV with many patients, and receives the model's estimated **probability of heart disease**, a colour-coded risk gauge, and a chart showing which inputs pushed that estimate up or down.

The project is organised so that the model, the feature pipeline and the user interface cannot drift apart silently. A single schema module defines every feature, and an automated test suite rebuilds the training notebook's preprocessing and asserts that the application produces an identical `(n, 31)` matrix.

Key fact	Value
Task	Binary classification: `has_heart_disease` (0 / 1)
Dataset	9,000 rows, 25 input features, 30.3% positive class (2,727 positive, 6,273 negative); no missing values, no duplicate rows
Model	Dense network 31 - 31 - 16 - 8 - 1, ReLU hidden layers, dropout 0.1, sigmoid output
Hold-out performance	Accuracy 0.896, precision 0.856, recall 0.801, specificity 0.939, F1 0.827 (n = 1,350)
Interface	Streamlit, four tabs, English and Persian with right-to-left layout
Engineering	Single-source schema, lazy TensorFlow import, pytest suite with a stub model, ruff lint, GitHub Actions CI, Docker image with health check
Runtime	Python 3.10 to 3.13 (upper bound set by TensorFlow)

Highlights

- **Complete, usable product.** Single assessment, batch scoring with downloadable output, what-if explanation, data explorer and an in-app model card.
- **Defensive engineering.** Strict input validation with human-readable errors, start-up consistency checks between model and pipeline, and CI that runs without TensorFlow.
- **Honest evaluation.** The report documents where the reported metrics are optimistic and where the model falls short (recall near 80%), instead of presenting only the headline accuracy.
- **Clear path forward.** Section 12 proposes five phases that take the project from a strong demo to a reproducible, calibrated and externally validated system.

> **Scope and intent**
> This is an educational and portfolio project. It is **not** a medical device and has not been clinically validated. Its numbers describe how a model behaves on one dataset, not how a patient will fare.

02 Background and Objectives

Cardiovascular disease is a major public-health concern, and early risk stratification from routine measurements is a classic application of supervised learning. Tabular clinical data, a binary outcome and a mix of numeric and categorical inputs make it a good vehicle for demonstrating the full machine-learning lifecycle: data understanding, modelling, evaluation, packaging and delivery.

Objectives

ID	Objective	How it is met
O1	Produce a calibrated-looking probability of heart disease from 25 inputs	Keras sigmoid model; probability shown as gauge and percentage
O2	Make the model usable by non-programmers	Streamlit form with grouped inputs, example profiles, bilingual UI
O3	Support scoring many patients at once	Batch tab with CSV template, strict validation and downloadable results
O4	Give users a sense of why a score is high or low	One-at-a-time what-if analysis shown as a diverging bar chart
O5	Prevent training/serving skew	Single-source schema and tests that rebuild the notebook pipeline
O6	Be easy to run, test and deploy	Dockerfile, health check, CI, pinned runtime ranges, documented setup

Non-goals

- Clinical diagnosis, treatment recommendation or any use with real patient data in care settings.
- Imaging, ECG or time-series signals: the model consumes a fixed vector of 25 tabular values.

03 Project Journey

The repository captures a complete path from raw data to a deployable application. Each stage leaves a concrete artefact that the next stage depends on.

Stage	What happened	Artefact
1. Data	Load the 9,000-row CSV, inspect types, balance and ranges; derive UI bounds and reference values	`data/Data.csv`
2. Preprocessing	Keep 22 numeric/boolean columns in CSV order; one-hot encode smoker status, chest-pain type and sex (3 + 4 + 2 = 9)	`OneHotEncoder.pkl`
3. Modelling	Train a dense network with early stopping and learning-rate reduction; 70 / 15 / 15 split	`Model.pkl`, `Config.pkl`
4. Evaluation	Accuracy, precision, recall, F1 and confusion matrix on the test split	Notebook outputs
5. Application	Schema-driven form, validation, predictor, charts, what-if analysis, bilingual strings	`app.py`, `heart_app/`
6. Quality	Tests with a stub model, ruff configuration, CI on Python 3.11 and 3.12	`tests/`, `.github/workflows/`
7. Delivery	Dockerfile with health check, Streamlit configuration, README and this report	`Dockerfile`, `.streamlit/`

04 Dataset

The training data is a single CSV with **9,000 records** and 26 columns: 25 input features and the binary target has_heart_disease. All values are present and no row is duplicated. The positive class represents 30.3% of records, a moderate imbalance that makes accuracy alone a misleading headline metric.

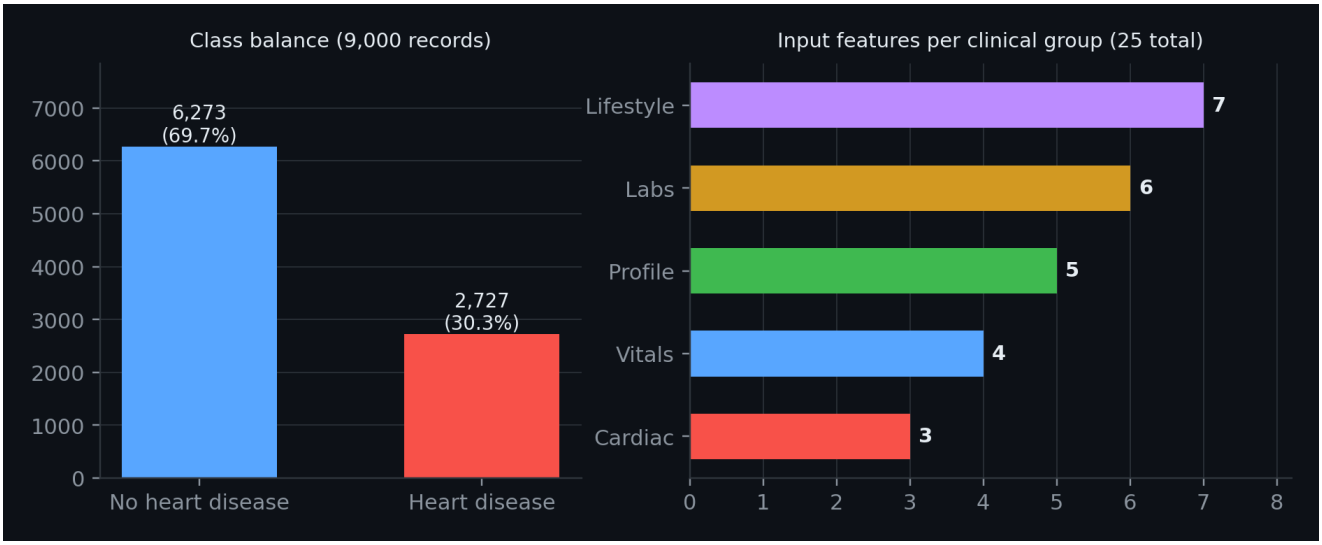


Figure 1. Class balance and feature groups.

Selected numeric summaries

Feature	Mean	Min	Max	Unit
age	54.0	18	90	years
resting_bp_systolic	127.8	85	181	mmHg
cholesterol_total	188.8	90	314	mg/dL
hdl	55.2	18	110	mg/dL
ldl	103.3	35	207	mg/dL
hba1c	5.8	4	8.6	%
bmi	25.3	15	43.3	kg/m2
max_heart_rate_achieved	164.8	93	210	bpm
st_depression	1.0	0	6.5	mm
daily_steps	6163.7	500	13950	steps
diet_quality_score	59.5	4.8	100	score

Exploratory feature importance

A Random Forest fitted in the notebook ranked max_heart_rate_achieved first (about 0.24), followed by exercise_induced_angina (about 0.12) and st_depression (about 0.07). This is an exploratory view from one run and is not a clinical finding.

Data provenance is undocumented

The repository does not describe where the dataset came from or how it was collected. Until that is established, conclusions should be treated as properties of this dataset only.

05 System Architecture

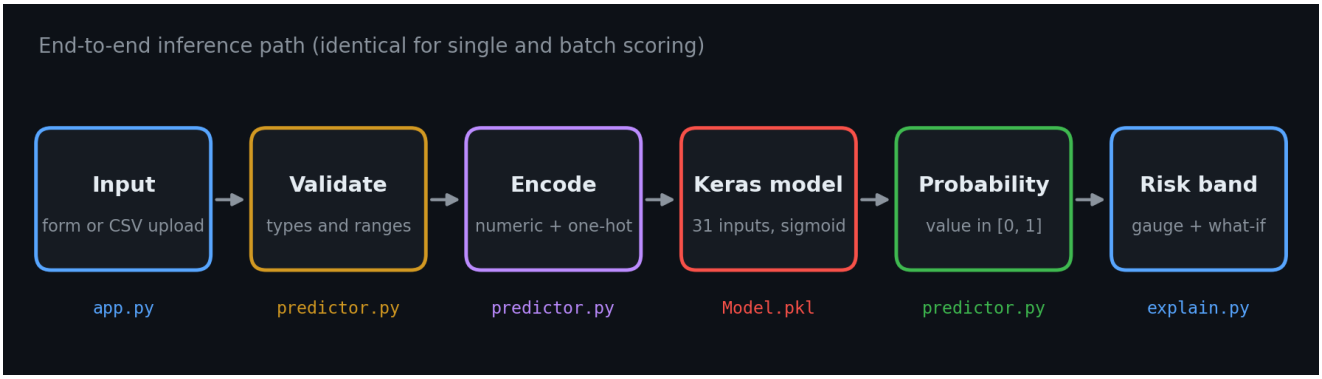


Figure 2. Inference path shared by single-patient and batch scoring.

Modules

File	Responsibility
`app.py`	Streamlit entry point: sidebar, language switch, four tabs, session state, CSV upload and download
`heart_app/schema.py`	The single source of truth: every feature's type, UI range, default, labels, help text, group, example profiles and risk bands
`heart_app/predictor.py`	Artefact loading, validation, type casting, feature construction and prediction; lazy TensorFlow import
`heart_app/explain.py`	One-at-a-time what-if contributions computed in a single batched model call
`heart_app/charts.py`	Plotly risk gauge and diverging contribution chart
`heart_app/i18n.py`	English and Persian strings, including right-to-left support

Design principles

- **One schema, many consumers.** The form, validator, presets and tests all derive from the same feature definitions, so adding or renaming a feature is a one-file change.
- **Lazy TensorFlow.** Importing the predictor does not import TensorFlow. The model is materialised only in `HeartRiskPredictor.load()`, which keeps tests and CI fast and light.
- **Fail loudly.** At start-up the model's `INPUT_DIM` is compared with the number of features the app builds; a mismatch raises immediately instead of producing silently wrong scores.
- **Training/serving parity.** Column order and one-hot ordering are locked by tests against the real dataset.

Technology stack

Layer	Technology	Purpose
Language	Python 3.10 - 3.13	Application and tests
UI	Streamlit 1.50+	Interactive web interface and caching
Model	Keras 3 on TensorFlow 2.19+	Neural network training and inference
Preprocessing	scikit-learn, pandas, NumPy	One-hot encoding, tabular handling
Visualisation	Plotly	Gauge, contribution chart, data explorer
Quality	pytest, ruff	Testing and linting
Delivery	Docker, GitHub Actions	Container image and CI

06 Feature Engineering and Model

Input vector

Block	Columns	Count
Numeric and boolean	22 columns in the exact order of the CSV header (age, blood pressure, lipids, glucose, HbA1c, BMI, heart rates, ST depression, angina, family history, lifestyle scores, and so on)	22
One-hot: smoker status	Never, Former, Current	3
One-hot: chest-pain type	Asymptomatic, Atypical Angina, Non-Anginal Pain, Typical Angina	4
One-hot: sex	Female, Male	2
Total		**31**

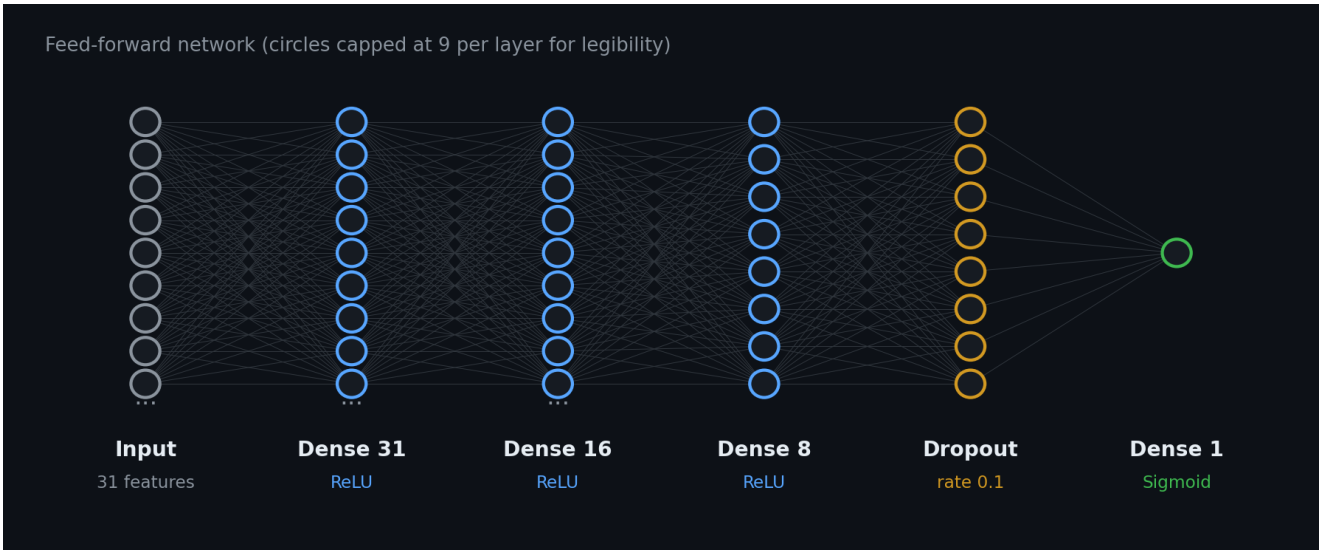


Figure 3. Network topology: 31 - 31 - 16 - 8 with dropout, then a single sigmoid unit.

Training configuration

Item	Value
Loss and optimiser	Binary cross-entropy, Adam
Callbacks	`EarlyStopping(monitor=val_loss, patience=50, restore_best_weights=True)` and `ReduceLROnPlateau(factor=0.4, min_lr=3e-4)`
Epochs	Maximum 500; stopped at epoch 137 with best weights from epoch 87
Split	70% train (6,300), 15% test (1,350), 15% validation (1,350)
Output	Probability of `has_heart_disease`

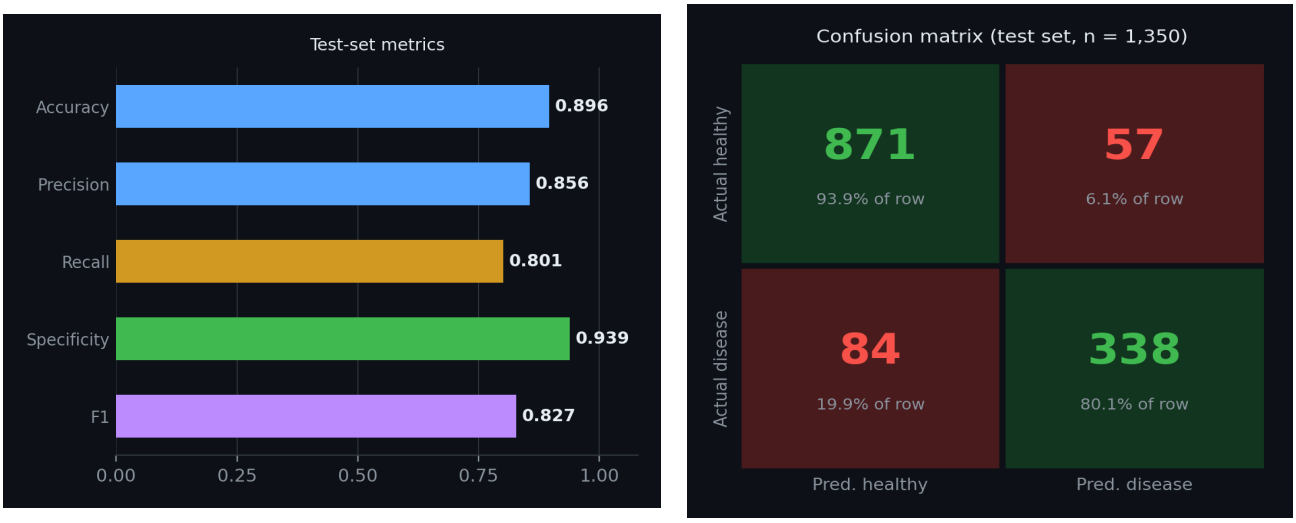


Figure 4. Test-set metrics and confusion matrix (n = 1,350).

Metric	Train	Test	Reading
Accuracy	0.8995	0.8956	Close to train: little visible overfitting
Precision	-	0.8557	338 of 395 flagged patients truly had disease
Recall (sensitivity)	-	0.8009	84 of 422 patients with disease were missed (19.9%)
Specificity	-	0.9386	57 of 928 healthy patients were flagged (6.1%)
F1	-	0.8274	Harmonic mean of precision and recall
Negative predictive value	-	0.9120	Reassurance carried by a 'healthy' prediction
Test prevalence	-	31.3%	Share of positives in the test split

Interpretation

The model is conservative: it is very good at confirming health (specificity 93.9%) and fairly precise when it raises an alarm (85.6%), but it misses about one in five patients with disease. For any screening-style use that trade-off would need to be shifted, for example by threshold tuning or class weighting, and then re-measured.

Risk bands used in the interface

Band	Probability	Meaning
Low	below 30%	Illustrative UI threshold
Moderate	30% to 60%	Illustrative UI threshold
High	60% and above	Illustrative UI threshold

The bands are presentation choices to make a probability easier to read; they are not clinically derived cut-offs.

07 Application

Tab	What it does
Assessment	Grouped form (Profile, Vitals, Labs, Cardiac, Lifestyle); one-click example profiles (typical, healthy, at-risk); probability gauge with risk band; what-if contribution chart
Batch prediction	Download a CSV template, upload up to 20 MB, validate, score every row, review summary metrics, download the results
Data explorer	Class balance, per-feature distributions split by outcome, correlation heatmap
About the model	Architecture, the exact 31-element input vector, training setup and limitations, inside the app

Validation rules for batch input

- All 25 feature columns must be present; extra columns (including the target) are ignored.
- No empty cells; numeric columns must contain numbers.
- Boolean columns accept `true/false`, `1/0`, `yes/no`, `y/n` and `t/f`, case-insensitive.
- Categorical columns must exactly match the categories seen in training; otherwise the error lists the allowed values.

User experience details

- Full English and Persian interface with Vazirmatn typeface and right-to-left styling.
- Every input has a unit, a plausible range and a help text, all defined once in the schema.
- Telemetry is disabled and entered data lives only in the Streamlit session; uploaded CSV files are processed in memory.

08 Explainability: What-If Analysis

For a single patient the app answers a practical question: *how much would the estimated risk change if this one value were typical?* The method is model-agnostic and fast.

Step	Operation
1	Compute the patient's baseline probability `p0`.
2	For each of the 25 features, create a variant in which only that feature is replaced by its reference value (dataset median or mode, stored as `Feature.default`).
3	Score all 26 inputs (baseline plus 25 variants) in one batched model call.
4	Compute `delta = p0 - p_variant`. Positive means the patient's value raises the estimate relative to a typical patient.

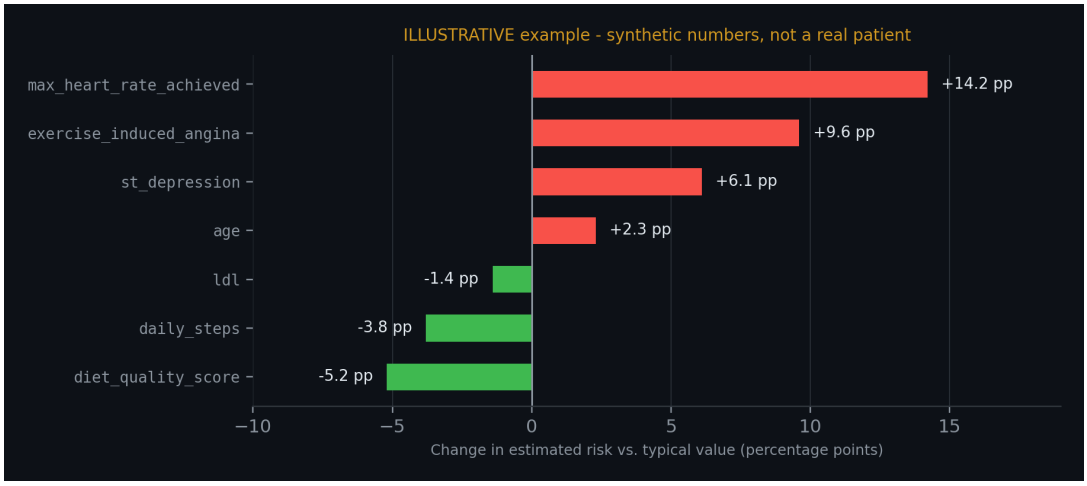


Figure 5. Illustrative contribution chart. Red bars push risk up; green bars push it down. Values are synthetic.

> **What this is and is not**
> It is a transparent approximation of the model's behaviour. It ignores interactions between features and says nothing about medical causation. Section 12 proposes adding SHAP values for a more rigorous attribution.

09 Engineering Quality

Automated checks

- Schema matches the CSV header exactly.
- The application's feature matrix equals the notebook pipeline's output, shape `(n, 31)`.
- Every default and example profile lies inside the declared ranges and covers every feature.
- Single and batch predictions return correct shapes and valid probabilities.
- String booleans are accepted; invalid input raises the right `PredictionInputError`.
- What-if contributions are sorted by magnitude and vanish for the reference profile.
- Risk bands map correctly.

Tests use a deterministic stub model, so the whole suite runs in seconds without TensorFlow. Continuous integration runs `ruff check .` and `pytest` on Python 3.11 and 3.12 for every push to `main` and every pull request.

Quality scorecard

Practice	Status	Note
Linting (ruff)	**Done**	Rules E, F, I, B, UP; line length 100
Unit tests	**Done**	Stub model, no TensorFlow required
Continuous integration	**Done**	GitHub Actions, Python 3.11 and 3.12
Container image	**Done**	Slim base, health check, KERAS_BACKEND preset
Input validation	**Done**	Strict, with explicit allowed values in errors
Fixed random seeds	Gap	Notebook sets none, so retraining is not bit-reproducible
Dependency lockfile	Gap	Version ranges only; pin exact training versions
Static type checking	Gap	No mypy or pyright step yet
Coverage reporting	Gap	Not measured in CI
Safe artefact format	Gap	Pickle files; see Section 11
Model card and data sheet	Gap	Not yet written

10 Deployment and Operations

Run locally

```
git clone https://github.com/<your-username>/Heart_attack_predictor.git
cd Heart_attack_predictor
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py        # http://localhost:8501
```

Run in Docker

```
docker build -t heart-risk-app .
docker run --rm -p 8501:8501 heart-risk-app
```

Topic	Detail
Streamlit Community Cloud	Push to GitHub, create a new app from `app.py`, and select Python 3.12 under advanced settings.
Other platforms	Any container platform (Cloud Run, Fly.io, Azure Container Apps, ECS) can run the image; expose port 8501.
Health check	The image probes `/_stcore/health`.
Configuration	`.streamlit/config.toml` sets the theme, a 20 MB upload limit and disables usage statistics.
Case-sensitive file systems	The dataset path is `data/Data.csv` (lowercase). On Linux, Docker and Streamlit Cloud the folder must be named exactly `data/`.
Version compatibility	Model saved with Keras 3.15.1 and encoder with scikit-learn 1.9.0. If loading fails, pin the training versions in `requirements.txt`.

11 Risks and Limitations

Reading the numbers responsibly matters more than the numbers themselves. The register below lists known weaknesses, how severe they are for the intended educational use, and how they would be addressed.

ID	Risk	Severity	Mitigation
R1	Clinical misuse of an unvalidated model	**High**	Prominent disclaimers; no storage of personal data; model card; never present output as diagnosis
R2	Optimistic evaluation: early stopping and learning-rate scheduling monitored the same split later reported as test	**High**	Use a separate validation set for training decisions and keep a truly untouched test set (Phase 1)
R3	Unverified data provenance and possible distribution shift	**High**	Document the source; validate on an independent dataset (Phase 5)
R4	Recall about 80%: roughly one in five positive cases missed	**High**	Threshold tuning, class weights, cost-sensitive evaluation (Phase 2)
R5	Uncalibrated probabilities: '70%' is a score, not a 70% chance	**Medium**	Platt or isotonic calibration, reliability plot, Brier score (Phase 3)
R6	Pickle artefacts can execute code when loaded	**Medium**	Load only trusted files; migrate to .keras and skops (Phase 4)
R7	Library version drift between training and serving	**Medium**	Pin exact versions and add a lockfile (Phase 1)
R8	No random seed in the notebook, so results vary between runs	**Low**	Fix seeds and log environment details (Phase 1)
R9	Config records dropout 0.01 while the model uses 0.1	**Low**	Align the recorded hyper-parameters with the architecture
R10	Folder case mismatch (Data/ vs data/) broke the data explorer on Linux	**Resolved**	Folders renamed to data/ and notebooks/

Responsible-use statement

Explanations are approximations and the model has not been audited for fairness across sexes, ages or other subgroups. A high score on a single dataset says little about real-world performance. For any health concern, consult a qualified clinician.

12 Roadmap

The roadmap below is a **proposal**. Phases overlap deliberately, and durations are indicative estimates for a single developer working part-time. Each phase ends with concrete deliverables and measurable success criteria.

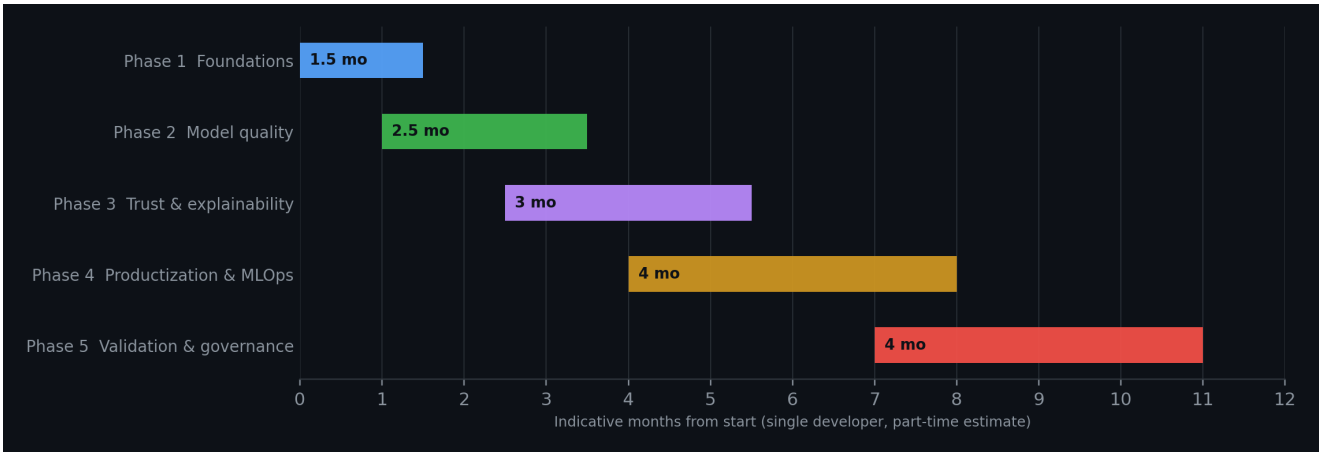


Figure 6. Proposed phases and indicative timeline.

Phase 1 Foundations and evaluation hygiene	
Goal	Make results trustworthy and reproducible before improving them.
Tasks	• Create a stratified train / validation / test split and use the validation set for early stopping and scheduling • Fix random seeds for Python, NumPy and Keras; log library versions • Align Config.pkl with the architecture (dropout 0.1) • Export exact dependency versions to a lockfile; add a CI badge and screenshots to the README
Deliverables	Reproducible training script or notebook, lockfile, corrected hyper-parameter record.
Success criteria	Re-running training twice gives metrics within a small, documented tolerance; test metrics come from data never used for any training decision.

Phase 2 Model quality	
Goal	Establish whether the neural network is actually the right model, and improve recall.
Tasks	• Add baselines: logistic regression, random forest and gradient boosting • Stratified k-fold cross-validation with confidence intervals (bootstrap) • Hyper-parameter search; class weights or focal loss • Decision-threshold analysis using precision-recall and cost-sensitive views
Deliverables	Comparison table of models with uncertainty, tuned threshold, updated confusion matrix.
Success criteria	Recall improves materially at an acceptable precision, and the chosen model beats the strongest baseline or the project explains why it is kept.

Phase 3 Trust and explainability	
Goal	Make probabilities meaningful and explanations rigorous.
Tasks	• Calibrate probabilities (Platt or isotonic); add reliability diagram and Brier score • Add SHAP attributions next to the what-if chart • Subgroup analysis by sex and age band; document gaps • Write a model card and a data sheet
Deliverables	Calibration report, SHAP view in the app, fairness notes, model card.
Success criteria	Calibration error is reported and reduced; each documented subgroup has metrics shown; limitations are visible inside the app.

Phase 4 Productization and MLOps	
Goal	Remove avoidable operational and security risk.
Tasks	• Migrate artefacts to .keras and skops to eliminate pickle loading • Expose the same schema as a FastAPI service with pydantic validation • Experiment tracking and versioning (MLflow, DVC or similar) • CI upgrades: coverage, type checking, dependency and security scanning, pre-commit hooks • Docker Compose for app plus API; basic monitoring of input drift
Deliverables	Safe artefact format, REST API, tracked experiments, hardened pipeline.
Success criteria	No pickle in the runtime path; CI fails on type errors or vulnerable dependencies; API and UI return identical predictions on the test set.

Phase 5 Validation and governance	
Goal	Test the model against reality and formalise responsible use.
Tasks	• Document dataset provenance; evaluate on an independent, external dataset • Seek review from a clinical advisor on features, thresholds and wording • Privacy and security review of upload handling • Accessibility pass and optional additional languages
Deliverables	External validation report, governance checklist, reviewed wording.
Success criteria	Performance on external data is reported honestly; the README, app and model card consistently describe intended and prohibited uses.

13 Appendices

A. Input features (25)

Group	Feature	Type	Range / values
Profile	age	int	18 - 90
Profile	sex	category	Female, Male
Profile	bmi	float	15.0 - 43.3
Profile	family_history	bool	True / False
Profile	smoker_status	category	Never, Former, Current
Vitals	resting_bp_systolic	int	85 - 181 mmHg
Vitals	resting_bp_diastolic	int	50 - 126 mmHg
Vitals	resting_heart_rate	int	48 - 111 bpm
Vitals	max_heart_rate_achieved	int	93 - 210 bpm
Labs	cholesterol_total	int	90 - 314 mg/dL
Labs	hdl	int	18 - 110 mg/dL
Labs	ldl	int	35 - 207 mg/dL
Labs	triglycerides	int	35 - 390 mg/dL
Labs	fasting_blood_sugar	int	60 - 204 mg/dL
Labs	hba1c	float	4.0 - 8.6 %
Cardiac	chest_pain_type	category	Asymptomatic, Atypical Angina, Non-Anginal Pain, Typical Angina
Cardiac	exercise_induced_angina	bool	True / False
Cardiac	st_depression	float	0.0 - 6.5 mm
Lifestyle	alcohol_units_per_week	float	0.0 - 45.9
Lifestyle	exercise_minutes_per_week	int	0 - 366
Lifestyle	sleep_hours	float	3.1 - 11.0
Lifestyle	stress_score	float	0 - 100
Lifestyle	wearable_owner	bool	True / False
Lifestyle	daily_steps	int	500 - 13,950
Lifestyle	diet_quality_score	float	4.8 - 100

B. Repository structure

```
.
|-- app.py                      Streamlit entry point
|-- heart_app/
|   |-- schema.py               features, ranges, presets, risk bands
|   |-- predictor.py            loading, validation, feature engineering
|   |-- explain.py              what-if contributions
|   |-- charts.py               Plotly gauge and contribution chart
|   `-- i18n.py                 English / Persian strings
|-- artifacts/                  Model.pkl, OneHotEncoder.pkl, Config.pkl
|-- data/Data.csv               training data (9,000 rows)
|-- notebooks/                  training notebook
|-- tests/test_predictor.py     pytest suite
|-- .streamlit/config.toml      theme, upload limit, telemetry off
|-- .github/workflows/ci.yml    ruff + pytest on Python 3.11 / 3.12
|-- Dockerfile  pyproject.toml  requirements.txt  requirements-dev.txt
`-- LICENSE  README.md
```

C. Developer commands

```
pip install -r requirements-dev.txt
ruff check .        # lint
pytest -q           # tests (no TensorFlow needed)
```

D. Glossary

Term	Meaning
Recall (sensitivity)	Share of truly positive patients that the model flags.
Specificity	Share of truly healthy patients that the model correctly clears.
Precision	Share of flagged patients that are truly positive.
F1 score	Harmonic mean of precision and recall.
Calibration	Agreement between predicted probabilities and observed frequencies.
One-hot encoding	Representing a category as a set of 0/1 columns, one per category.
Early stopping	Halting training when a monitored metric stops improving.
What-if analysis	Re-scoring a patient with one value replaced to see its effect.
Pickle	Python serialisation format that can execute code when loaded.

E. Reproducibility checklist

- Use the same Keras and scikit-learn versions that created the artefacts.
- Keep preprocessing order: non-categorical columns in CSV order, then one-hot of smoker status, chest-pain type, sex.
- After retraining, replace all three artefacts together and run the test suite.
- Record seeds, split indices and library versions with every reported metric.